

Laporan Praktikum dan Tugas Mandiri Machine Learning



Muhammad Riandana Pranatha - 0110224076

Teknik Informatika, STT Terpadu Nurul Fikri. Depok

0110224076@student.nurulfikri.ac.id

Abstrak

Pada praktikum ini dilakukan penerapan algoritma Support Vector Machine (SVM) untuk menyelesaikan permasalahan klasifikasi menggunakan dua dataset berbeda, yaitu Iris dan Heart Disease. Dataset Iris digunakan untuk memahami konsep dasar klasifikasi SVM dengan data yang sederhana dan mudah divisualisasikan, sedangkan dataset Heart Disease digunakan sebagai penerapan lanjutan untuk kasus klasifikasi medis berbasis data nyata dari Kaggle.

Proses yang dilakukan mencakup eksplorasi data awal, pemisahan data menjadi training dan testing, pelatihan model SVM menggunakan kernel linear, serta evaluasi performa model melalui akurasi, classification report, dan confusion matrix. Hasil dari kedua percobaan menunjukkan bahwa algoritma SVM mampu menghasilkan performa yang baik dalam membedakan kelas-kelas target pada masing-masing dataset.

Secara keseluruhan, praktikum ini membuktikan bahwa metode Support Vector Machine dapat diterapkan secara efektif untuk permasalahan klasifikasi baik pada data sederhana seperti Iris maupun data medis yang lebih kompleks seperti Heart Disease.

1. Menghubungkan Google Colab dengan Google Drive

Langkah pertama adalah menyambungkan Colab ke akun Google Drive agar file dataset dapat diakses. Setelah menjalankan perintah otorisasi, pengguna akan diminta login dan memberikan izin. Proses ini hanya perlu dilakukan sekali pada setiap sesi Colab.

```
5] 18 d from google.colab import drive
    drive.mount('/content/drive')

Mounted at /content/drive

5] 0 d path = "/content/drive/MyDrive/Praktikum//praktikum06/Data/"
```

1.2 Membaca Dataset CSV

Dengan menggunakan library Pandas, dataset dalam format .csv dimuat ke dalam DataFrame. Hal ini memudahkan manipulasi serta analisis data. Perintah head() dipakai untuk menampilkan beberapa baris pertama sehingga struktur data bisa langsung terlihat.

```
d
import pandas as pd

df = pd.read_csv(path + 'Iris.csv')

df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

1.3 Melihat Informasi Dataset

Perintah info() menampilkan ringkasan struktur data, seperti jumlah baris, nama kolom, tipe data, serta banyaknya nilai non-null. Dari sini kita bisa mengetahui apakah ada data kosong dan bagaimana karakteristik tipe datanya.

```
1 df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column             Non-Null Count  Dtype  
---  -
 0   Id                  150 non-null   int64  
 1   SepalLengthCm       150 non-null   float64
 2   SepalWidthCm        150 non-null   float64
 3   PetalLengthCm       150 non-null   float64
 4   PetalWidthCm        150 non-null   float64
 5   Species             150 non-null   object  
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

1.4 Eksplorasi Statistik Deskriptif Dataset

Kode ini menampilkan ringkasan statistik dari setiap kolom numerik dalam dataset, seperti jumlah data, rata-rata, standar deviasi, nilai minimum, maksimum, dan kuartil.

Tujuannya untuk melihat sebaran data dan mendeteksi kemungkinan nilai yang tidak normal (outlier).

```
1 df.describe()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

1.5 Mengecek Nilai Unik pada Kolom Target

Kode ini digunakan untuk melihat daftar kategori unik yang ada pada kolom Species.

Dengan begitu, kita bisa tahu kelas atau label apa saja yang akan diprediksi oleh model (misalnya: Setosa, Versicolor, dan Virginica pada dataset Iris).

```
df["Species"].unique()
array(['Iris-setosa', 'Iris-versicolor', 'Iris-virginica'], dtype=object)
```

1.6 Mengecek Jumlah Data per Kelas

Kode ini menampilkan jumlah data pada setiap kategori di kolom Species. Tujuannya untuk memastikan apakah dataset seimbang atau tidak — misalnya, apakah jumlah data tiap spesies (Setosa, Versicolor, Virginica) sama banyak.

Hal ini penting supaya model tidak bias terhadap salah satu kelas.

```
df["Species"].value_counts()

```

Species	count
Iris-setosa	50
Iris-versicolor	50
Iris-virginica	50

dtype: int64

1.7 Menentukan Fitur dan Label (Variabel Independen dan Dependen)

Kode ini memisahkan fitur (X) dan label (y) dari dataset.

- X berisi kolom fitur yang akan digunakan untuk melatih model (panjang dan lebar sepal serta petal).

- y berisi kolom target, yaitu nama spesies bunga yang ingin diprediksi.

Langkah ini penting agar model tahu mana data yang digunakan sebagai input dan mana yang menjadi output (hasil prediksi).

```
8] 0 d X = df[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']]
    y = df['Species']
```

1.8 Melihat Contoh Data Fitur dan Label

Kode ini menampilkan 5 baris pertama dari data fitur (X) dan label (y). Tujuannya untuk memastikan bahwa pemisahan data sudah benar — kolom fitur berisi nilai numerik (panjang & lebar sepal/petal), sedangkan kolom label menampilkan nama spesies bunga.

X.head()

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

```
[ ] y.head()

Species
0  Iris-setosa
1  Iris-setosa
2  Iris-setosa
3  Iris-setosa
4  Iris-setosa

dtype: object
```

1.9 Membagi Data dan Melatih Model SVM

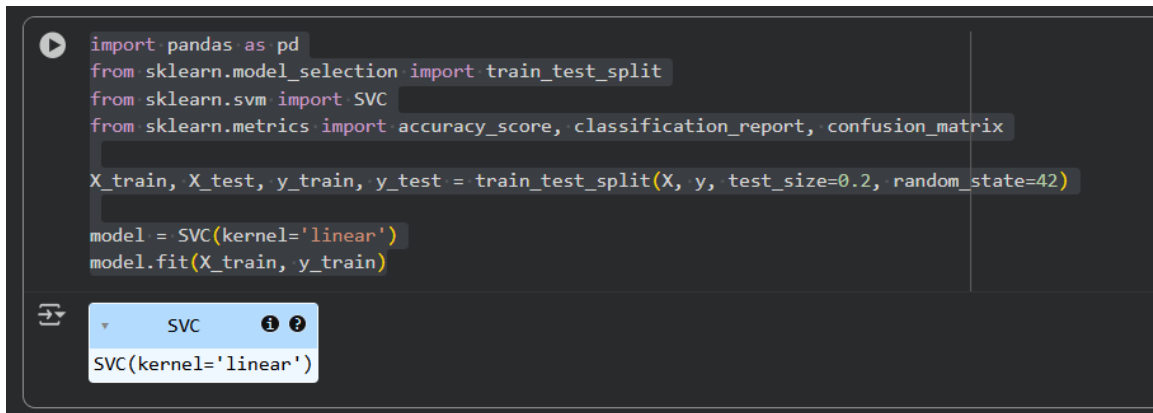
Bagian ini melakukan dua hal penting:

1. Membagi dataset → Data dibagi menjadi training set (80%) dan testing set (20%) menggunakan `train_test_split()`.
 - `random_state=42` digunakan agar hasil pembagian selalu sama setiap kali dijalankan.
2. Melatih model SVM (Support Vector Machine) → Model dibuat dengan `SVC(kernel='linear')` dan dilatih menggunakan `fit(X_train, y_train)`.
Kernel linear digunakan karena cocok untuk data yang bisa dipisahkan secara garis lurus di ruang fitur.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = SVC(kernel='linear')
model.fit(X_train, y_train)
```



2. Evaluasi Model SVM

Kode ini digunakan untuk mengevaluasi performa model SVM setelah dilatih.

- `model.predict(X_test)` menghasilkan prediksi kelas berdasarkan data uji.
- `accuracy_score()` menghitung persentase prediksi yang benar dari seluruh data uji.
- `classification_report()` menampilkan metrik evaluasi seperti:
 - Precision → ketepatan prediksi model,
 - Recall → seberapa baik model menemukan kelas sebenarnya,
 - F1-score → keseimbangan antara precision dan recall.


```
[ ] y_pred = model.predict(X_test)

print(f"Akurasi: {accuracy_score(y_test, y_pred) * 100:.2f}%")

print("\nLaporan Klasifikasi:\n", classification_report(y_test, y_pred))
```

Akurasi: 100.00%

Laporan Klasifikasi:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	1.00	1.00	1.00	9
Iris-virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

2.1 Visualisasi Confusion Matrix

Kode ini digunakan untuk melihat hasil prediksi model dalam bentuk confusion matrix.

- `confusion_matrix()` menampilkan jumlah prediksi yang benar dan salah untuk tiap kelas.
- Visualisasinya dibuat menggunakan heatmap dari seaborn agar lebih mudah dibaca.
 - Sumbu X menunjukkan label prediksi,
 - Sumbu Y menunjukkan label sebenarnya.
 Semakin tinggi nilai diagonal (warna lebih gelap), semakin baik model dalam memprediksi kelas dengan benar.

```

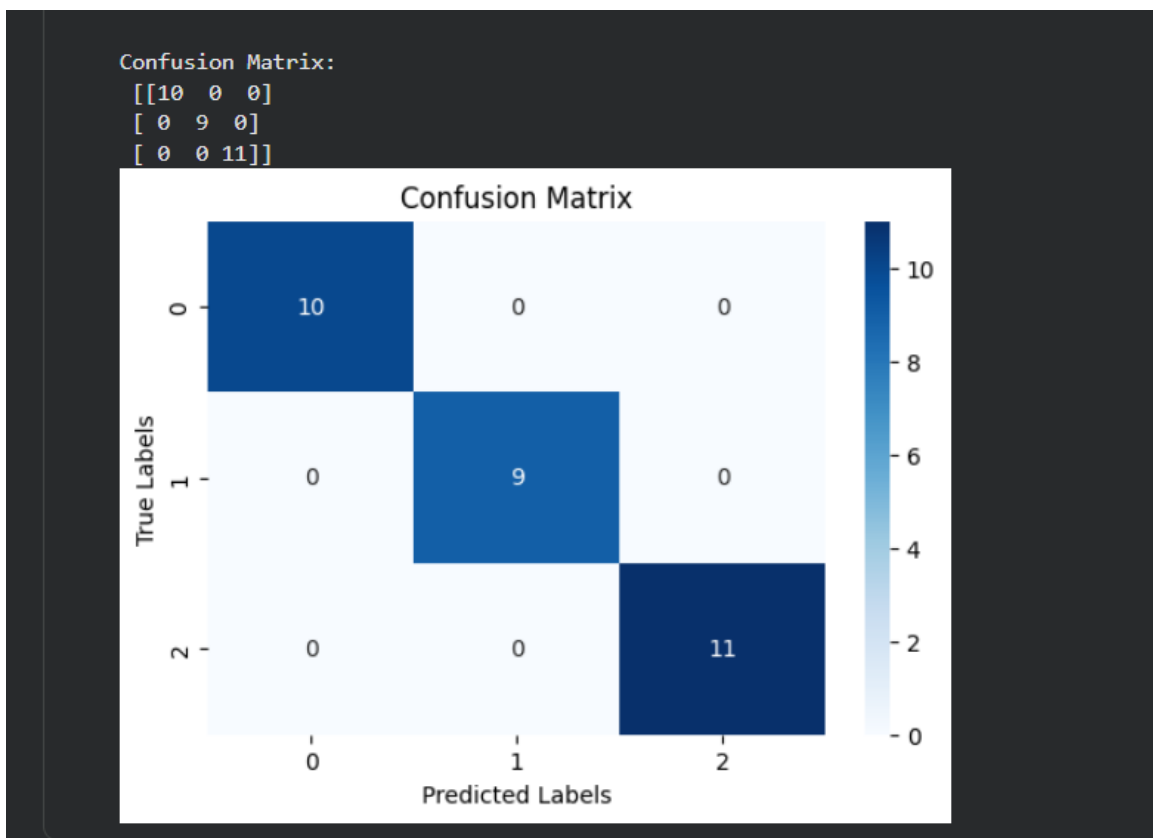
1 from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
import seaborn as sns

print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted Labels")
plt.ylabel("True Labels")
plt.show()

```



2.2 Visualisasi Sebaran Data Berdasarkan Kelas

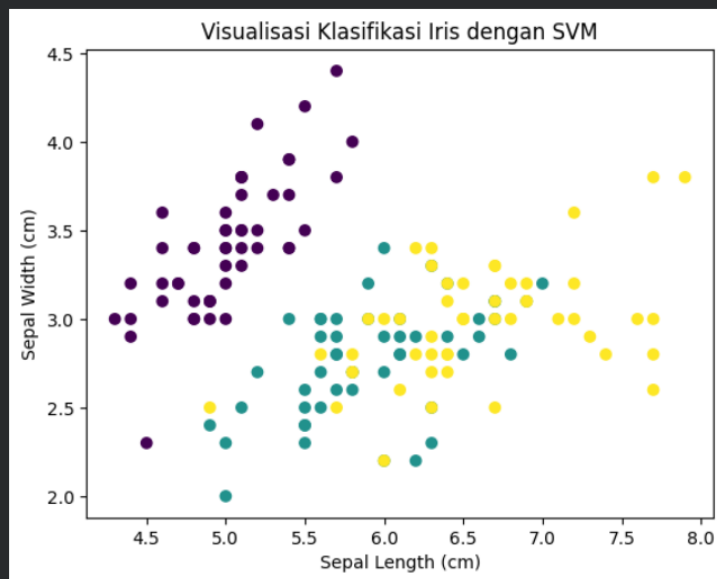
Kode ini menampilkan visualisasi sebaran data (scatter plot) dari dataset Iris.

- Setiap titik mewakili satu bunga, dengan posisi ditentukan oleh panjang dan lebar sepal.

- Warna titik diatur berdasarkan jenis spesies (Species) menggunakan kode numerik (cat.codes).
Tujuannya untuk melihat apakah data tiap spesies memiliki pola atau batas pemisahan yang jelas — hal ini membantu memahami bagaimana SVM memisahkan tiap kelas.

```
import matplotlib.pyplot as plt

plt.scatter(df['SepalLengthCm'], df['SepalWidthCm'], c=df['Species'].astype('category').cat.codes)
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.title('Visualisasi Klasifikasi Iris dengan SVM')
plt.show()
```



2.3. Visualisasi Data dalam 3 Dimensi

Kode ini digunakan untuk menampilkan visualisasi 3D dari dataset Iris berdasarkan tiga fitur utama:

- Sepal Length (cm) → sumbu X,
- Sepal Width (cm) → sumbu Y,
- Petal Width (cm) → sumbu Z.

Langkah-langkah penting:

1. LabelEncoder() mengubah nama spesies menjadi angka agar bisa digunakan untuk pewarnaan.
2. ax.scatter() membuat plot 3D dengan warna berbeda untuk setiap spesies (r, g, b).
3. Grafik ini membantu melihat pemisahan antar kelas secara visual di ruang tiga dimensi.

```
1 from sklearn.preprocessing import LabelEncoder
  from sklearn.metrics import accuracy_score
  from mpl_toolkits.mplot3d import Axes3D

  le = LabelEncoder()
  df['SpeciesEncoded'] = le.fit_transform(df['Species'])

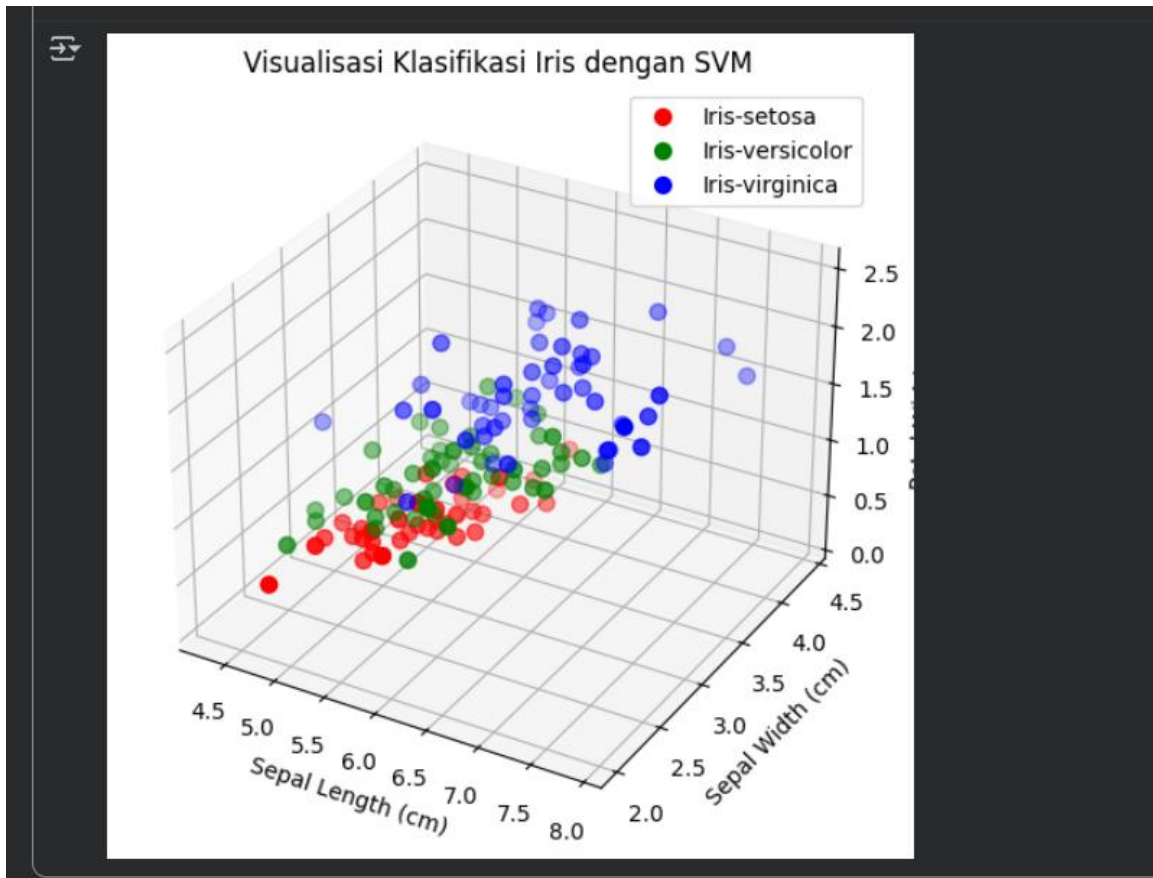
  fig = plt.figure(figsize=(10, 6))
  ax = fig.add_subplot(111, projection='3d')

  colors = ['r', 'g', 'b']
  labels = le.classes_

  for i, species in enumerate(labels):
      subset = df[df['SpeciesEncoded'] == i]
      ax.scatter(
          subset['SepalLengthCm'],
          subset['SepalWidthCm'],
          subset['PetalWidthCm'],
          c=colors[i],
          label=species,
          s=50
      )

  ax.set_xlabel('Sepal Length (cm)')
  ax.set_ylabel('Sepal Width (cm)')
  ax.set_zlabel('Petal Width (cm)')
  ax.set_title('Visualisasi Klasifikasi Iris dengan SVM')
  ax.legend()

  plt.show()
```



3. Tugas Mandiri

3.1 Instruksi

Mencari dataset di Kaggle, untuk Solusi klasifikasi menggunakan SVM.

3.2 Langkah Penyelesaian

- Melakukan import dataset sekaligus melakukan langkah awal seperti menyambungkan Gdrive dan membaca dataset, serta menjalankan `df.head()`.

```
import pandas as pd

df = pd.read_csv(path + 'heart.csv')

df.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

- Memuat dataset heart.csv ke dalam DataFrame dan melakukan eksplorasi awal menggunakan fungsi describe(), unique(), serta value_counts() untuk memahami struktur data dan distribusi kelas target.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column        Non-Null Count  Dtype  
---  -
 0   age           1025 non-null   int64  
 1   sex           1025 non-null   int64  
 2   cp            1025 non-null   int64  
 3   trestbps      1025 non-null   int64  
 4   chol          1025 non-null   int64  
 5   fbs           1025 non-null   int64  
 6   restecg       1025 non-null   int64  
 7   thalach       1025 non-null   int64  
 8   exang         1025 non-null   int64  
 9   oldpeak       1025 non-null   float64 
10   slope         1025 non-null   int64  
11   ca            1025 non-null   int64  
12   thal          1025 non-null   int64  
13   target        1025 non-null   int64  
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

```
df.describe()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000
mean	54.434146	0.695610	0.942439	131.611707	246.000000	0.149268	0.529756	149.114146	0.336585	1.071512	1.385366	0.754146	2.323902	0.513171
std	9.072290	0.460373	1.029641	17.516718	51.59251	0.356527	0.527878	23.005724	0.472772	1.175053	0.617755	1.030798	0.620660	0.500070
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000	71.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	48.000000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000	132.000000	0.000000	0.000000	1.000000	0.000000	2.000000	0.000000
50%	56.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.000000	152.000000	0.000000	0.800000	1.000000	0.000000	2.000000	1.000000
75%	61.000000	1.000000	2.000000	140.000000	275.000000	0.000000	1.000000	166.000000	1.000000	1.800000	2.000000	1.000000	3.000000	1.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000	202.000000	1.000000	6.200000	2.000000	4.000000	3.000000	1.000000

Kode ini menampilkan ringkasan statistik dari setiap kolom numerik dalam dataset Heart, seperti jumlah data, rata-rata, standar deviasi, serta nilai minimum dan maksimum.

Tujuannya untuk memahami sebaran nilai tiap fitur seperti age, kolesterol, dan thalach.

```
[9] df.columns
Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'], dtype='object')
```

```
[10] df["target"].unique()
array([0, 1])
```

```
[11] df["target"].value_counts()
count
target
1      526
0      499
dtype: int64
```

Kode ini menampilkan nilai unik dari kolom target, yaitu kolom target yang berisi 0 atau 1.

- 0 → tidak berpotensi penyakit jantung,
- 1 → berpotensi atau memiliki penyakit jantung.

Menampilkan jumlah data pada setiap kelas target.

Hal ini untuk memastikan distribusi data seimbang antara pasien yang memiliki dan tidak memiliki penyakit jantung.

- Menentukan fitur (X) dan label (y), kemudian membagi dataset menjadi data training (80%) dan testing (20%) menggunakan `train_test_split`.

```
[12] X = df[['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs',
✓ 0 d      'restecg', 'thalach', 'exang', 'oldpeak',
        'slope', 'ca', 'thal']]

      y = df['target']
```

```
[14] X.head()
✓ 0 d
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2

Langkah berikutnya: [Buat kode dengan X](#) [New interactive sheet](#)

```
[16] y.head()
✓ 0 d
```

	target
0	0
1	0
2	0
3	0
4	0

dtype: int64

[Bagaimana cara menginstal library Python?](#) [Muat data dari Go](#)

Apa yang bisa saya bantu buat untuk Anda?

Kode ini memisahkan:

- X (fitur): berisi data numerik seperti umur, tekanan darah, kolesterol, dan detak jantung.
 - y (label): berisi kolom target (0 atau 1) sebagai hasil yang ingin diprediksi model.
- Membuat model SVM dengan kernel linear dan melatihnya menggunakan data training melalui `fit(X_train, y_train)`.


```

18] y_pred = model.predict(X_test)
0 d
print(f"Akurasi: {accuracy_score(y_test, y_pred) * 100:.2f}%")

print("\nLaporan Klasifikasi:\n", classification_report(y_test, y_pred))

```

Akurasi: 80.49%

Laporan Klasifikasi:

	precision	recall	f1-score	support
0	0.88	0.71	0.78	102
1	0.76	0.90	0.82	103
accuracy			0.80	205
macro avg	0.82	0.80	0.80	205
weighted avg	0.82	0.80	0.80	205

Data dibagi menjadi:

- 80% untuk pelatihan (training)
- 20% untuk pengujian (testing)

Kemudian model SVM dengan kernel linear dilatih untuk membedakan pasien yang berisiko penyakit jantung dan yang tidak.

- Melakukan evaluasi model dengan menghitung akurasi, serta menampilkan classification report dan confusion matrix untuk menilai kinerja prediksi.

```

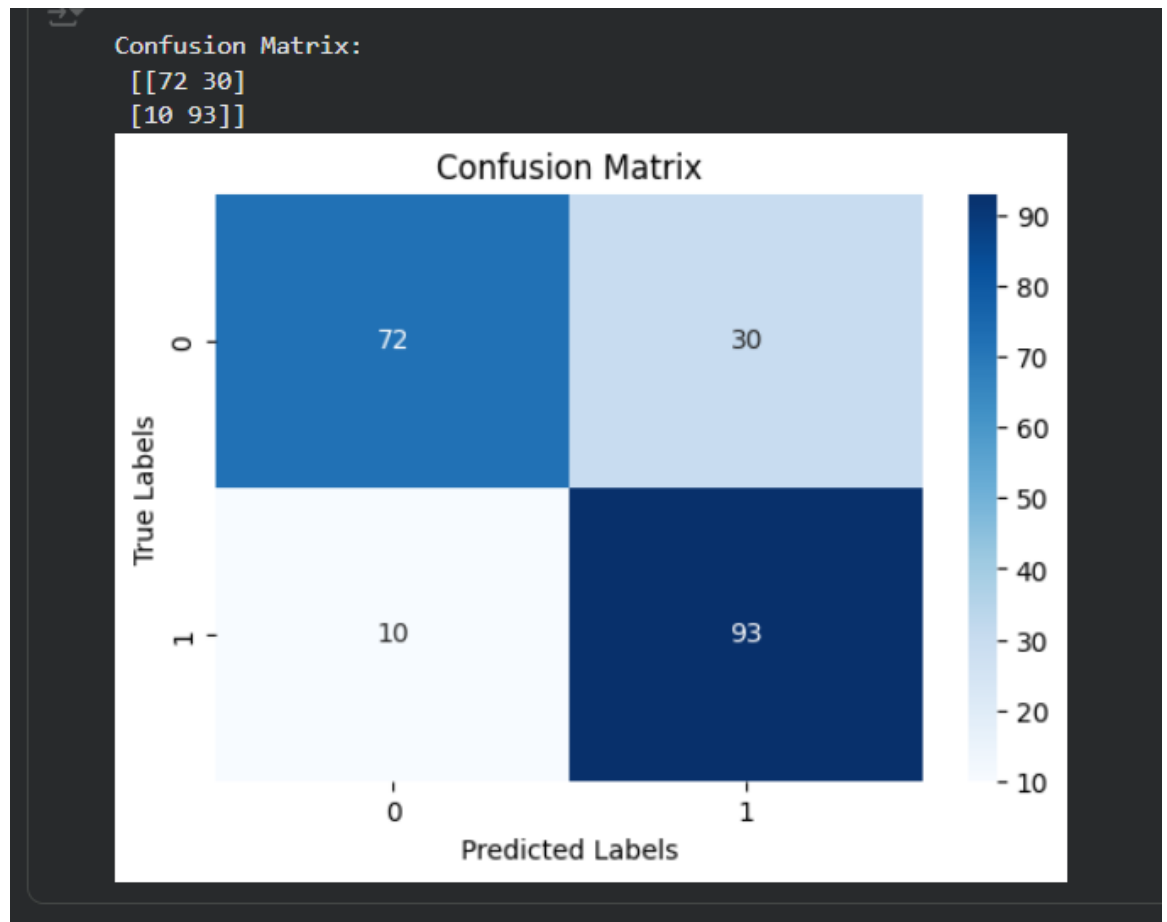
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
import seaborn as sns

print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted Labels")
plt.ylabel("True Labels")
plt.show()

```

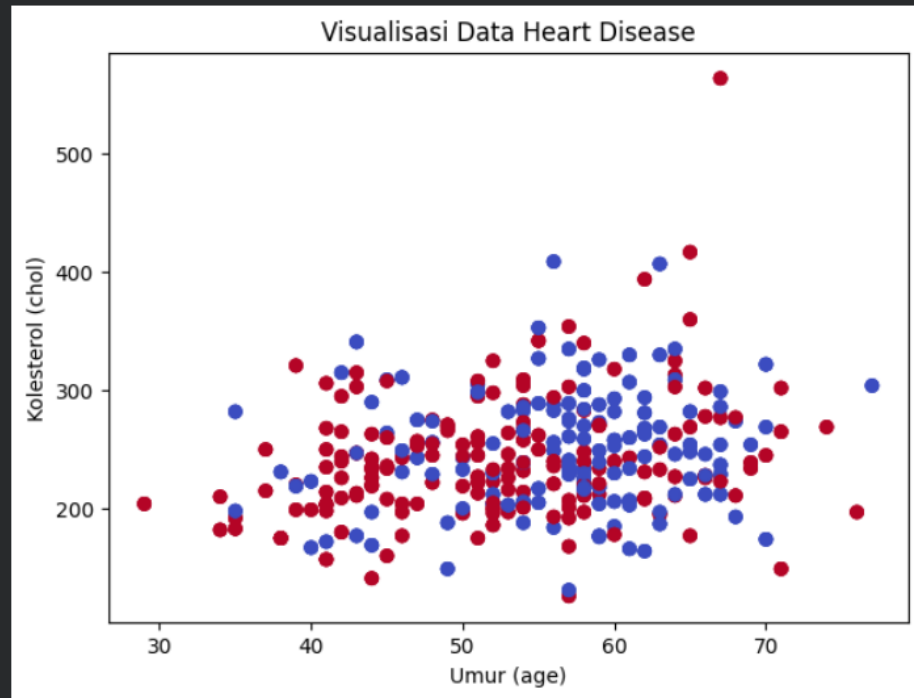


Confusion Matrix digunakan untuk melihat hasil prediksi model. Semakin besar nilai diagonal, semakin akurat model dalam memprediksi pasien dengan benar.

- Membuat visualisasi sebaran data dalam bentuk 2D dan 3D untuk melihat pola hubungan antar fitur seperti usia, kolesterol, dan detak jantung maksimum terhadap status penyakit jantung.

```
21]  
1d
```

```
import matplotlib.pyplot as plt  
  
plt.figure(figsize=(7,5))  
plt.scatter(df['age'], df['chol'], c=df['target'], cmap='coolwarm')  
plt.xlabel('Umur (age)')  
plt.ylabel('Kolesterol (chol)')  
plt.title('Visualisasi Data Heart Disease')  
plt.show()
```



Scatter plot ini menampilkan hubungan antara umur (age) dan detak jantung maksimum (thalach).

Warna titik menunjukkan status target (0 = sehat, 1 = sakit).

Dari plot ini bisa terlihat perbedaan pola antara pasien yang sehat dan berisiko.

- Menarik kesimpulan dari hasil visualisasi dan evaluasi bahwa model SVM mampu mengklasifikasikan kondisi pasien dengan cukup baik berdasarkan data yang diberikan.

2]
0 d



```
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt

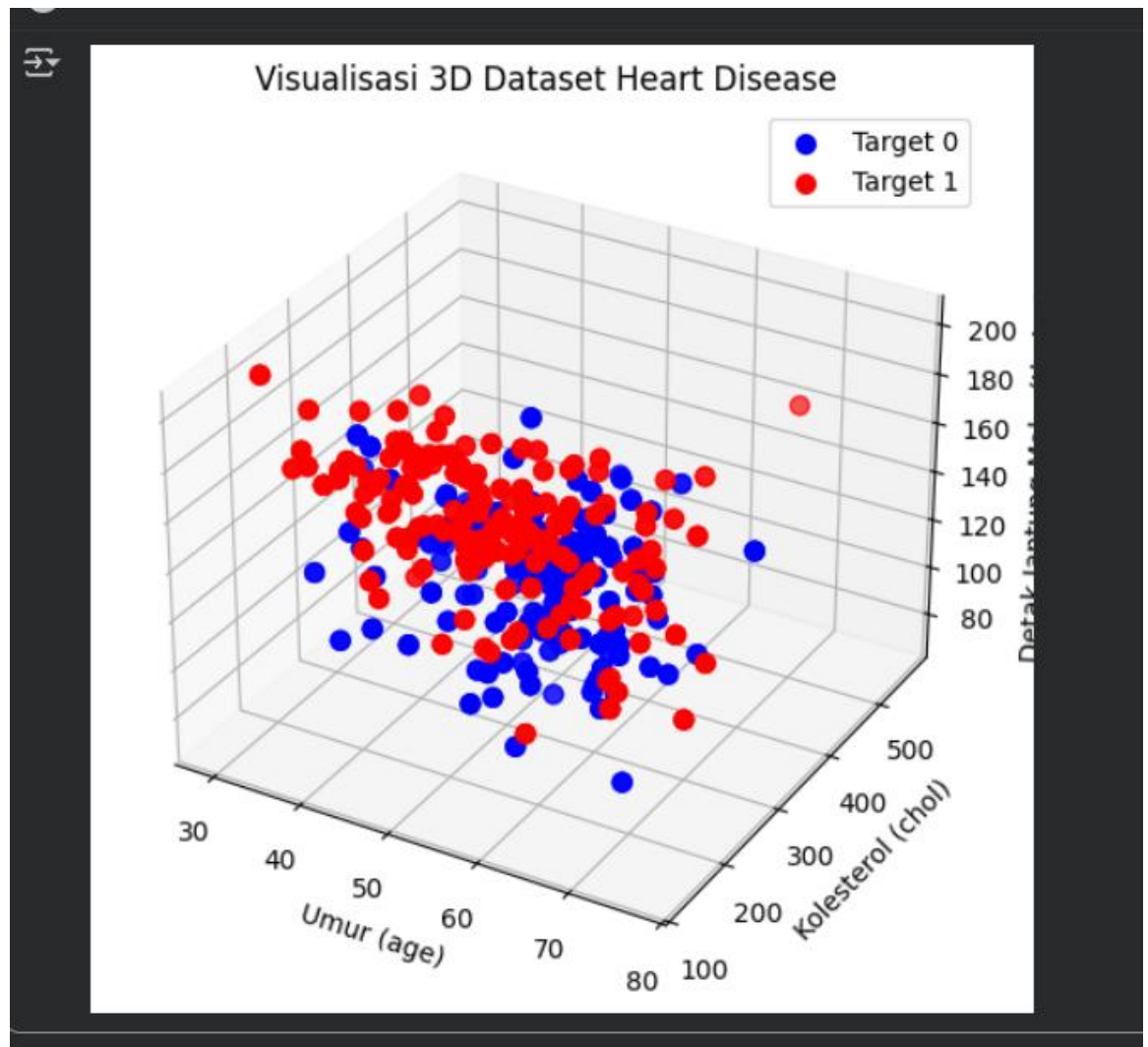
fig = plt.figure(figsize=(10, 6))
ax = fig.add_subplot(111, projection='3d')

colors = {0: 'b', 1: 'r'}

for label in df['target'].unique():
    subset = df[df['target'] == label]
    ax.scatter(
        subset['age'],
        subset['chol'],
        subset['thalach'],
        c=colors[label],
        label=f"Target {label}",
        s=50
    )

ax.set_xlabel('Umur (age)')
ax.set_ylabel('Kolesterol (chol)')
ax.set_zlabel('Detak Jantung Maks (thalach)')
ax.set_title('Visualisasi 3D Dataset Heart Disease')
ax.legend()

plt.show()
```



Plot 3D ini menunjukkan hubungan antara umur, kolesterol, dan detak jantung maksimum, dengan pewarnaan berdasarkan status penyakit jantung.

Visualisasi ini membantu memahami pola distribusi data dalam ruang tiga dimensi.

Kesimpulan

Berdasarkan hasil praktikum klasifikasi menggunakan algoritma Support Vector Machine (SVM) pada dataset Heart Disease, dapat disimpulkan bahwa model mampu melakukan klasifikasi dengan tingkat akurasi yang cukup baik. Melalui tahapan eksplorasi data, pembagian dataset, pelatihan model, hingga evaluasi, diperoleh gambaran bahwa fitur-fitur seperti usia,

tekanan darah, kolesterol, dan detak jantung maksimum memiliki pengaruh besar terhadap prediksi kondisi penyakit jantung.

Hasil evaluasi yang ditunjukkan melalui akurasi, classification report, serta confusion matrix membuktikan bahwa model SVM dengan kernel linear dapat memisahkan kelas pasien dengan dan tanpa penyakit jantung secara efektif. Visualisasi 2D dan 3D juga membantu memperjelas pola data antar kelas.

Secara keseluruhan, implementasi SVM pada dataset ini menunjukkan bahwa metode ini cocok digunakan untuk permasalahan klasifikasi medis berbasis data numerik, terutama dalam mendeteksi potensi penyakit jantung sejak dini.

Berikut Link Github dan Google Drive:

Github : <https://github.com/Sakiaihara12/Machine-Learning.git>

Drive:

https://drive.google.com/drive/folders/1z4s6VxDRpkNs1RD_qpknpSDESOR8IMQ2?usp=sharing

Dataset Kaggle: <https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset>